

DRY-RUN GUIDE | check_prime.js

Source: javascript\2nd_belt\check_prime.js

CORE BEHAVIOR

Defines two primality functions; the top-level examples call the simple isPrime version. It rejects ≤ 1 , then checks divisors through $\text{sqrt}(\text{num})$.

REPRESENTATIVE INPUT / CALL

Fixed calls: `isPrime(11)`, `isPrime(4)`, `isPrime(1)`.

STEP TRACE

11 has no divisor 2..3 -> true; 4 is divisible by 2 -> false; 1 is rejected by ≤ 1 .

EXPECTED OUTPUT / RETURN

Lines: 'true', 'false', 'false'.

ACTUAL SOURCE EXCERPT

Verbatim lines from the source; long lines wrap. Source files were not modified.

```
001 | // Question: Check whether a number is prime.
002 | // Add your solution here.
003 |
004 | function isPrimeOptimized(num) {
005 |     if (num <= 1) return false;
006 |     if (num <= 3) return true; // 2 and 3 are prime
007 |
008 |     // Exclude all even numbers and multiples of 3
009 |     if (num % 2 === 0 || num % 3 === 0) return false;
010 |
011 |     // Check odd numbers up to the square root, skipping multiples of 2 and 3
012 |     for (let i = 5; i * i <= num; i += 6) {
013 |         if (num % i === 0 || num % (i + 2) === 0) return false;
014 |     }
015 |
016 |     return true;
017 | }
018 | // or
019 | function isPrime(num) {
020 |     // 1. Numbers less than or equal to 1 are not prime
021 |     if (num <= 1) return false;
022 |
023 |     // 2. Check for factors up to the square root of the number
024 |     for (let i = 2; i <= Math.sqrt(num); i++) {
025 |         if (num % i === 0) {
026 |             return false; // Found a factor, so it is not prime
027 |         }
028 |     }
029 |
030 |     return true; // No factors found, so it is prime
031 | }
032 |
033 | // Example usage:
034 | console.log(isPrime(11)); // true
035 | console.log(isPrime(4));  // false
036 | console.log(isPrime(1));  // false
037 |
```